

COURSE MAP - CONCEPT GUIDE - QUICK REFERENCE

Software Engineering

Course Map & Reference Handbook

HSBA - 2nd Semester - Based on the 2026 course slides

Version 1.0 - 6 August 2026

A progressive route from course logic to detailed terminology - designed for learning, comparison, and fast lookup.

Source boundary: local 2026 lecture decks 01-03 and 05-11 plus Video Club. Deck 04 and assessment rules remain open.

Contents

How to use this handbook	6
Evidence legend	6
Four levels of depth	6
Five ways to use the handbook.....	6
Course logic at a glance	7
The nine-step reasoning chain	7
The core artefact chain	8
Question navigator	9
1. Foundations: what software engineering contains	10
Core logic	10
Key terms	10
Important distinctions	11
Typical mistakes	11
MentorHive application	11
Exit criterion.....	11
2. Software quality: define what 'good' means	12
Core logic	12
Key terms	12
ISO 25010 model used in the course	13
Important distinctions	13
Typical mistakes	14
MentorHive application	14
Exit criterion.....	14
3. Requirements engineering: from need to testability	15
Core logic	15
Key terms	15
Important distinctions	16
Typical mistakes	16

MentorHive application	16
Exit criterion	16
4. Source gap: deck 04 is not available	17
Core logic	17
Important distinctions	17
Typical mistakes	17
MentorHive application	17
Exit criterion	17
5. Teams and organisational structures	18
Core logic	18
Key terms	18
Important distinctions	19
Typical mistakes	19
MentorHive application	20
Exit criterion	20
6. Software development processes: feedback and flow	21
Core logic	21
Key terms	21
Important distinctions	22
Typical mistakes	23
MentorHive application	23
Exit criterion	23
7. System architecture: decompose the technical system	24
Core logic	24
Key terms	24
Pattern reference	25
Important distinctions	26
Typical mistakes	26
MentorHive application	26
Exit criterion	26

8. Software architecture: structure, patterns, and decisions	27
Core logic	27
Key terms	27
Pattern reference	28
Important distinctions	28
Typical mistakes	29
MentorHive application	29
Exit criterion	29
9. Design and implementation: build quality in	30
Core logic	30
Key terms	30
Important distinctions	31
Typical mistakes	32
MentorHive application	32
Exit criterion	32
10. Builds and releases: make change traceable and releasable	33
Core logic	33
Key terms	33
DORA metrics	34
Important distinctions	34
Typical mistakes	35
MentorHive application	35
Exit criterion	35
11. Operations and maintenance: learn from reality	36
Core logic	36
Key terms	36
Important distinctions	37
Typical mistakes	37
MentorHive application	38
Exit criterion	38

12. End-to-end case: MentorHive.....	39
13. High-value comparisons.....	40
Functional requirement vs quality requirement vs acceptance criterion	40
Sequential and iterative process choices.....	40
System architecture vs software architecture	40
Test levels and feedback.....	41
CI, Continuous Delivery, and Continuous Deployment.....	41
Monitoring, observability, and incident learning	41
14. Video Club: targeted depth.....	42
15. Alphabetical glossary	43
16. Source map and open information	50
Open course information.....	50
One-page final recall	51

How to use this handbook

This document supports two reading modes: a progressive course route from broad logic to deep concepts, and a reference route from a question or term directly to the relevant chapter.

Evidence legend

Status	Meaning
COURSE SOURCE	A statement, model, or term is stated or visibly shown in the local lecture PDFs.
SYNTHESIS	A relationship, learning route, or worked example is derived from the sequence of several course sources.
OPEN	The available files do not determine the answer. No assumption is presented as a course fact.

Four levels of depth

Depth	What you should be able to do
Level 1 - Recognise	Name the concept and place it in the software lifecycle.
Level 2 - Explain	Define it in your own words and distinguish it from adjacent concepts.
Level 3 - Apply	Use it to create an artefact, analyse a scenario, or make a concrete decision.
Level 4 - Justify	Explain consequences and trade-offs across quality, requirements, organisation, architecture, delivery, and operations.

Five ways to use the handbook

Get the big picture: Read the course map and the concept chain first. They show why the topics appear in this order and how the output of one topic becomes the input of the next.

Learn progressively: Use the depth ladder: recognise the vocabulary, explain the distinctions, apply the models, then justify trade-offs across topics.

Look up a question: Start with the question navigator or the alphabetical glossary. Both point to the relevant chapter and the underlying course deck.

Check understanding: Each chapter ends with an exit criterion. If you can produce that result without the slides, you understand the chapter at an application level.

Trace the evidence: Every chapter contains a source trail with deck and slide/page ranges. PDF page numbers and slide numbers are identical in this source set.

Course logic at a glance

The lecture order is a learning sequence, not a waterfall delivery process. The deeper course logic is an iterative feedback system: define value and quality, organise work, structure the solution, deliver it, observe reality, and update earlier decisions.

THE COURSE BUILDS A FEEDBACK SYSTEM

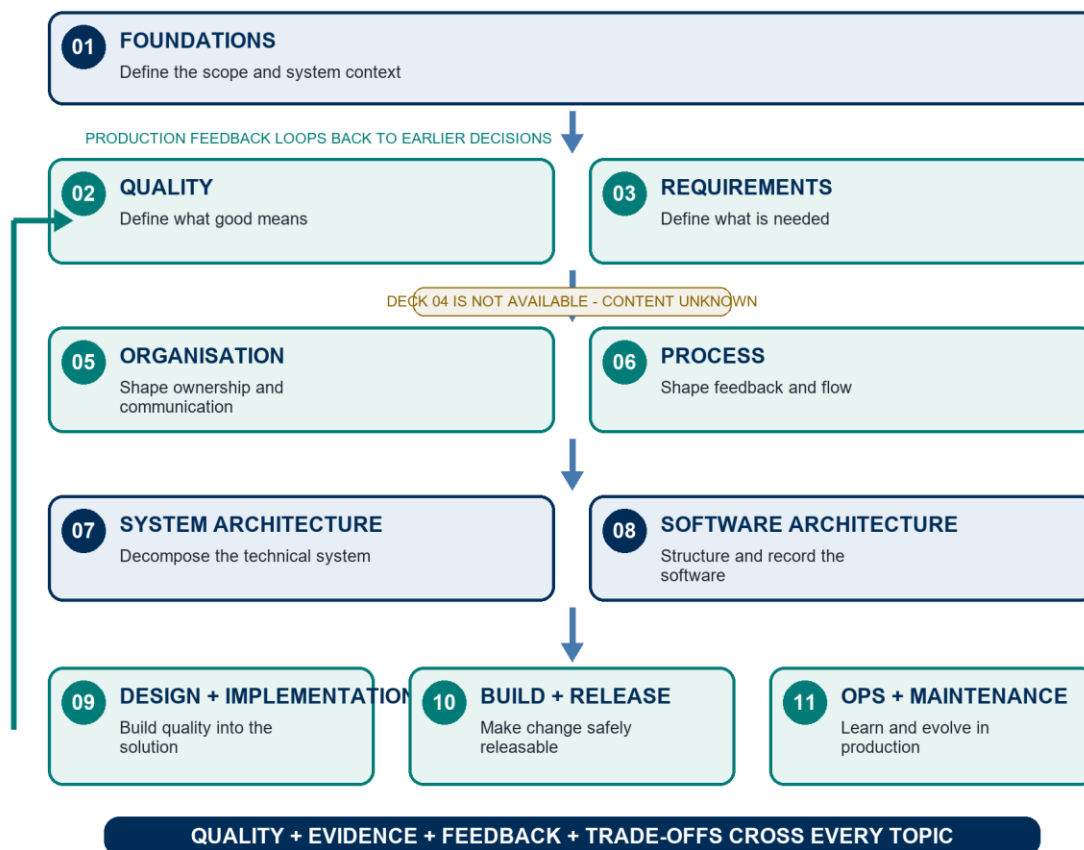


Figure 1. Source-grounded course sequence with a synthesis feedback loop.

CORE IDEA Software engineering is the management of decisions and feedback across the complete product lifecycle. Code is one artefact in that system, not the system itself.

The nine-step reasoning chain

- Start with the problem and the system context, not with code.
- Define required quality and functional behaviour before selecting solutions.
- Refine vague intent into measurable quality scenarios and testable acceptance criteria.
- Design the team structure and development process to create fast, reliable feedback.

- Decompose the system and record architecture decisions with their consequences.
- Implement the solution while building in quality through design systems, tests, reviews, and fault handling.
- Automate integration, build, release, and deployment so small changes remain traceable and releasable.
- Observe production, learn from incidents, and repay technical debt.
- Feed production evidence back into quality targets, requirements, process, and architecture.

The core artefact chain

Each artefact reduces a different uncertainty. The chain is synthesis: the individual artefacts are course-sourced, while the consolidated handoff view connects several decks.

Artefact	Question answered	Primary topic
Problem / product need	A user or business outcome worth solving	Discovery and product thinking
Quality scenarios	Measurable conditions and response targets	Software quality / NFRs
User stories	Role, need, and benefit	Functional requirements
Rules, examples, open questions	Shared refinement of a story	Example Mapping
Acceptance criteria	Binary, outcome-focused evidence of done	Requirements and BDD
Architecture decision record	Context, decision, status, and all consequences	Architecture
Design, code, and tests	Executable solution plus fast feedback	Implementation
Build artefact and release	Traceable, approved, deployable version	Build and release
Signals, incident record, action items	Evidence from the running system	Operations
Backlog and refactoring	Changed requirements and improved internal structure	Maintenance and next iteration

Question navigator

Start here when you know the question but not the course term.

Question	Concepts to look up	Go to
What does software engineering include beyond coding?	Scope, layers, lifecycle activities	Chapter 1
How do we define whether software is good enough?	ISO 25010, quality scenarios, trade-offs	Chapter 2
Functional or non-functional requirement?	FR, NFR, quality criterion	Chapter 3
How do we turn a vague need into something testable?	User Story, Example Mapping, acceptance criteria, Gherkin	Chapter 3
Why is a team not progressing smoothly?	Team characteristics, Tuckman stages	Chapter 5
Project or product organisation?	Project, product, triangle, organisation forms	Chapter 5
Which development process fits this situation?	Sequential, iterative, Waterfall, V-Model, Scrum, Kanban	Chapter 6
Why is everyone busy but delivery is still slow?	Utilisation, throughput, WIP, bottleneck, waste	Chapter 6
When is a story ready or genuinely done?	Definition of Ready, Definition of Done	Chapter 6
How should the system be split into parts?	Decomposition, cohesion, coupling, layering, partitioning	Chapter 7
Which architecture pattern should we consider?	Layered, N-tier, SOA, REST, Microservices, SCS, EDA, ETL, MVC	Chapter 7
System architecture or software architecture?	Technical-system boundary versus software structure	Chapters 7-8
How do we record an architecture choice?	ADR, UML, arc42	Chapter 8
Which design pattern solves this recurring code problem?	Singleton, Adapter, Facade, Mediator, Observer, Strategy	Chapter 8
Which type of test belongs where?	Testing Pyramid, test double, acceptance test, BDD	Chapter 9
Review, pair, or work as an ensemble?	Four-eye principle and feedback timing	Chapter 9
What is the difference between version, release, hash, and tag?	Version management and SemVer	Chapter 10
How do CI, Delivery, Deployment, and delivery metrics differ?	CI/CD, automated promotion, DORA metrics	Chapter 10
Monitoring or observability?	Known signals versus open-ended diagnosis	Chapter 11
How do we learn from incidents and improve legacy code?	Blameless PMA, refactoring strategies, technical debt	Chapter 11

1. Foundations: what software engineering contains

COURSE DECK 01

Software engineering is the disciplined coordination of many activities needed to create, deliver, operate, and evolve software. Programming is one activity inside that wider system.

GUIDING QUESTION Which layers, environments, and lifecycle activities must be considered before focusing on code?

Core logic

- Distinguish the software product from the hardware and infrastructure on which it runs.
- Recognise the software layers from infrastructure and data to backend, frontend, and user interface.
- Treat development process, quality, requirements, design, architecture, implementation, testing, delivery, maintenance, scaling, and operations as one connected discipline.
- Compare how real organisations perform these activities; there is no single company structure or process that fits every context.

Key terms

Term	Working definition
Software engineering	An engineering discipline covering the decisions and activities required to build, deliver, run, and evolve software - not just programming.
Infrastructure	The compute, network, runtime, and deployment foundation on which software operates.
Database	Persistent storage and retrieval of application data.
Backend	Server-side logic, data access, and service interfaces.
Frontend	Client-side software that presents information and handles interaction.
User Interface (UI)	The screens and controls through which a user interacts with a system.
On-premise data centre	Computing facilities controlled at an organisation's own site.
Cloud server	Remotely provisioned computing resources operated through a cloud environment.
Operations	Running, monitoring, and supporting software in production.
Maintenance	Correcting, adapting, and evolving software after delivery.

Important distinctions

- Frontend is the client-side application layer; UI is the visible interaction surface. They overlap but are not identical.
- Cloud, mobile, desktop, and on-premise describe target environments; they are not software layers.
- A historical nuance: the course credits Margaret Hamilton with using and legitimising the term, while the 1968 and 1969 NATO conferences helped popularise it.

Typical mistakes

- Reducing software engineering to coding or to a single development method.
- Reading the scope list as a fixed waterfall sequence. The later process deck explicitly presents repeated, iterative activity.
- Selecting tools or architecture before understanding the problem and quality needs.

MentorHive application

SYNTHESIS For MentorHive, the user-facing booking page is only the surface. It depends on frontend and backend logic, data storage, infrastructure, delivery automation, monitoring, and ongoing maintenance.

Exit criterion

CHECK YOURSELF Draw the full software landscape for one familiar product. Label layers, runtime environments, and at least eight lifecycle activities; explain one dependency between them.

SOURCE TRAIL Course source: SW Engineering - 01 - Intro - 2026, slides/pages 2-11. The lifecycle interpretation is synthesis; slide 10 itself is a scope list without arrows.

2. Software quality: define what 'good' means

COURSE DECK 02

Quality is a whole-team responsibility. It becomes useful only when abstract characteristics are prioritised, balanced, and expressed as observable, measurable scenarios.

GUIDING QUESTION Which qualities matter for this product, under which conditions, and how will we know that the required level has been reached?

Core logic

- Identify relevant quality characteristics rather than treating quality as one vague property.
- Prioritise by product context and risk; maximum quality in every dimension is neither realistic nor always valuable.
- Make trade-offs explicit, including the quality gained and the quality sacrificed.
- Express the target as a scenario containing conditions, an event, an observable response, and a measurable threshold.
- Use the scenarios to steer requirements, architecture, tests, monitoring, and maintenance.

Key terms

Term	Working definition
Whole Team Quality	Quality is owned across roles and activities, not delegated to testers or a QA department.
Functional Suitability	Functions cover the intended tasks, produce correct results, and support user objectives appropriately.
Performance Efficiency	Required response time, throughput, resource use, and capacity are achieved under specified conditions.
Compatibility	The product co-exists and exchanges usable information with other systems.
Interaction Capability	Specified users can understand and operate the system to complete tasks in their context of use; the slides also call it usability.
Reliability	The system performs required functions over time, remains available, tolerates faults, and can recover.
Security	The system protects information and resists malicious action according to authorised access.
Maintainability	The system can be analysed, tested, modified, and reused effectively without excessive side effects.
Flexibility	The product can adapt, scale, install, and replace components across changing contexts and environments.

Term	Working definition
Safety	The product avoids states that endanger life, health, property, or the environment.
Quality trade-off	A deliberate choice that improves one quality characteristic while accepting a consequence in another.
Quality scenario	A measurable statement linking conditions and a stimulus to an observable response and quality characteristic.
Overengineering	Adding more complexity, capacity, or quality than the context requires, thereby increasing cost and slowing change.

ISO 25010 model used in the course

The slides name nine characteristics and the following sub-characteristics. The edition of ISO 25010 is not stated in the deck.

Characteristic	Sub-characteristics named in the deck
Functional Suitability	Completeness; correctness; appropriateness
Performance Efficiency	Time behaviour; resource utilisation; capacity
Compatibility	Co-existence; interoperability
Interaction Capability	Recognisability; learnability; operability; user error protection; engagement; inclusivity; assistance; self-descriptiveness
Reliability	Faultlessness; availability; fault tolerance; recoverability
Security	Confidentiality; integrity; non-repudiation; accountability; authenticity; resistance
Maintainability	Modularity; reusability; analysability; modifiability; testability
Flexibility	Adaptability; scalability; installability; replaceability
Safety	Operational constraint; risk identification; fail safe; hazard warning; safe integration

Important distinctions

- Functional correctness is one quality sub-characteristic; it is not a synonym for overall quality.
- Performance is more than response time: it also covers throughput, resource utilisation, and capacity.
- Reliability is more than availability: fault tolerance and recoverability matter as well.
- A login message can trade user assistance for confidentiality by revealing less information to an attacker.

Typical mistakes

- Using adjectives such as fast, secure, scalable, or user-friendly without a condition and measurable target.
- Assuming every quality characteristic can be maximised simultaneously.
- Delegating quality to testers after implementation.
- Ignoring the long-term cost of low maintainability or the short-term cost of unnecessary complexity.

MentorHive application

SYNTHESIS MentorHive quality scenario: During peak evening traffic, when a learner searches for mentors, matching results are returned within two seconds for at least 95% of requests. This supports Performance Efficiency - Time Behaviour. The exact threshold is a synthesis value added for practice, not a stated course requirement.

Exit criterion

CHECK YOURSELF Create three prioritised quality scenarios for one feature. Each must name the context, event, observable response, target value, and ISO 25010 characteristic; explain one trade-off.

SOURCE TRAIL Course source: SW Engineering - 02 - Quality - 2026, slides/pages 2-26. ISO 25010 is named without an edition; do not infer a year from the slides.

3. Requirements engineering: from need to testability

COURSE DECK 03

Requirements engineering converts product intent into shared agreements. Functional requirements describe behaviour; non-functional requirements describe the required quality of that behaviour.

GUIDING QUESTION What does the user need, why does it matter, which rules apply, and what observable evidence proves that the requirement is fulfilled?

Core logic

- Separate functional behaviour from quality expectations, while keeping the two connected.
- State functional intent as a User Story from the user's perspective without prescribing UI or implementation.
- Refine the story collaboratively through rules, examples, open questions, and possible story splits.
- Resolve important questions before implementation starts.
- Define outcome-focused acceptance criteria that produce a clear pass or fail.
- Expect requirements to change, be reduced, or be discarded; detail only what will be built soon enough to avoid specification waste.

Key terms

Term	Working definition
Functional Requirement (FR)	A required feature or behaviour of the system.
Non-functional Requirement (NFR)	A measurable quality expectation for the system's behaviour.
User Story	A concise requirement stated as role, desired action, and benefit: As [role], I want [action], so that [outcome].
Refinement	Collaborative clarification, splitting, changing, or removal of requirements before implementation.
Example Mapping	A collaborative method that links one story to rules, examples, open questions, and newly discovered stories.
Rule	A condition or policy that governs how a User Story behaves.
Example	A concrete case illustrating how a rule applies, including boundaries and exceptions.
Open Question	An unresolved issue made visible during refinement and clarified no later than the start of development.

Term	Working definition
Acceptance Criterion	A specific, outcome-focused condition that can be verified as pass or fail and describes when a story is done.
Gherkin	A Given-When-Then format: Given context, When an event occurs, Then an observable outcome follows.
Specification waste	Detailed requirement work that becomes obsolete before the team needs to implement it.

Important distinctions

- A User Story defines the need and benefit; acceptance criteria define evidence of completion.
- Example Mapping discovers detail and uncertainty; it is not a substitute for deciding the final acceptance criteria.
- Acceptance criteria can use bullets or Gherkin. Gherkin is a useful format, not the only valid one.
- Non-functional does not mean optional; quality constraints can be decisive for product success or safety.

Typical mistakes

- Writing technical or UI instructions inside the User Story and closing the design space too early.
- Omitting the benefit in the 'so that' clause.
- Refining alone rather than building shared understanding with the team.
- Writing acceptance criteria that describe implementation instead of observable outcomes.
- Starting development while critical open questions remain unresolved.

MentorHive application

SYNTHESIS User Story: As a learner, I want to book an available slot with a matched mentor so that I can receive guidance. Rule: a slot can be booked only once. Example: two learners confirm the same slot almost simultaneously; exactly one booking succeeds. Open question: which time zone controls availability? Acceptance criterion: Given an available slot, when the learner confirms it, then one booking is created and both learner and mentor receive a confirmation.

Exit criterion

CHECK YOURSELF Take one vague product statement and produce a complete refinement set: User Story, at least two rules, three examples including an edge case, open questions, and two pass/fail acceptance criteria.

SOURCE TRAIL Course source: SW Engineering - 03 - Requirements Engineering - 2026, slides/pages 2-19. The repeated MentorHive pitch links stories, Example Mapping, and acceptance criteria.

4. Source gap: deck 04 is not available

COURSE DECK 04

The local source sequence jumps from deck 03 to deck 05. The topic, learning objectives, and possible assessment information for deck 04 are unknown.

GUIDING QUESTION What must remain explicitly open rather than being invented?

Core logic

- Do not infer the missing topic merely from the neighbouring filenames.
- Use the confirmed decks as the course boundary for this handbook.
- Add deck 04 only after the original file or an official course description becomes available.

Important distinctions

- A logical bridge created in this handbook is synthesis, not evidence of the missing deck's content.

Typical mistakes

- Renumbering the later decks and hiding the source gap.
- Inventing a likely topic, exam rule, or learning objective for deck 04.

MentorHive application

SYNTHESIS The course map preserves the missing number and continues with deck 05. This makes the evidence boundary visible without interrupting the learning route.

Exit criterion

CHECK YOURSELF Explain which parts of the handbook are course facts, which are synthesis, and which remain open because deck 04 is absent.

SOURCE TRAIL *Open: no local file named SW Engineering - 04 is present as of 6 August 2026.*

5. Teams and organisational structures

COURSE DECK 05

Software delivery is a socio-technical activity. Shared goals, complementary skills, communication, accountability, and organisational boundaries shape both flow and architecture.

GUIDING QUESTION How should people, responsibilities, and communication be organised for the product and its constraints?

Core logic

- Build a team around a common goal, collaboration, communication, complementary skills, interdependence, and accountability.
- Recognise the team's development stage and adapt leadership rather than assuming stable high performance.
- Distinguish temporary projects from long-lived products and operations.
- Manage scope, time, and budget as connected constraints; quality suffers when one changes without compensation elsewhere.
- Choose a structural form and a procedural form; responsibilities and recurring workflows both matter.
- Anticipate how communication structures will influence the resulting system design.

Key terms

Term	Working definition
Team	People with a common goal who collaborate through complementary skills, communication, interdependence, and shared as well as individual accountability.
Tuckman model	A model of team development: Forming, Storming, Norming, Performing, and Adjourning; membership changes may reset the team.
Project	A temporary, unique endeavour with a defined beginning and end, managed within constraints such as scope, time, and budget.
Product	A value offering managed across its lifecycle from idea through development, launch, operation, and evolution.
Project Management Triangle	Scope, time, and budget constrain the work; uncompensated change in one dimension reduces quality or forces change in another.
Structural organisation	The allocation of roles, responsibilities, hierarchy, positions, and reporting lines.
Procedural organisation	The recurring processes and coordination rules through which work moves.

Term	Working definition
Functional organisation	People are grouped by specialist department, supporting expertise but creating cross-department handoffs.
Project organisation	People leave or cross their units to form a dedicated temporary delivery team.
Matrix organisation	People retain a functional home while contributing to one or more product or project teams, creating dual alignment.
Spotify model	A matrix vocabulary of autonomous Squads, capability-focused Chapters, collections of Squads called Tribes, and learning communities called Guilds.
Holacracy	An organisation of autonomous circles and explicit roles connected through governance roles.
Conway's Law	Systems tend to reflect the communication structures of the organisations that design them.

Important distinctions

- Structural organisation answers who owns and reports; procedural organisation answers how recurring work and coordination happen.
- A project ends; a product continues to create value and therefore requires long-term ownership, operations, and maintenance.
- A matrix can combine product delivery with long-term capability development, but line and product priorities may conflict.

Typical mistakes

- Treating quality as an independent fourth project constraint that remains unchanged while scope, time, or budget is compressed.
- Expecting a changed team to continue immediately at the Performing stage.
- Copying branded organisation models without understanding the communication and responsibility problem they address.
- Ignoring organisational handoffs when designing services and system boundaries.

MentorHive application

SYNTHESIS MentorHive benefits from a cross-functional product team that owns the learner journey end to end, with product, design, engineering, quality, and infrastructure capabilities. The system boundaries should support that ownership rather than force every change through several functional departments.

Exit criterion

CHECK YOURSELF Choose an organisational form for a product scenario. Explain its communication paths, likely priority conflicts, and two architecture consequences through Conway's Law.

SOURCE TRAIL Course source: SW Engineering - 05 - Organisational Structure - 2026, slides/pages 2-23. Project/product definitions, Tuckman stages, organisation forms, Spotify vocabulary, and Holacracy are explicit; the selection guidance is synthesis.

6. Software development processes: feedback and flow

COURSE DECK 06

A development process orders recurring engineering activities and creates shared waypoints. The core choice is not 'process or no process' but how much sequencing, iteration, feedback, and flow the context requires.

GUIDING QUESTION How should work move so the team learns quickly, controls risk, and finishes valuable outcomes rather than accumulating started work?

Core logic

- Recognise the repeated activities: discovery, requirements, design, implementation, testing, delivery, validation, operations, and maintenance.
- Choose an appropriate relationship between sequential planning and iterative learning.
- Use roles, events, artefacts, and team boards to create shared coordination.
- Optimise throughput and completed value, not maximum individual utilisation.
- Limit work in progress, expose bottlenecks, and swarm to restore flow.
- Use a Definition of Ready to prevent avoidable waiting and a Definition of Done to prevent invisible nearly-finished work.
- Scale only after understanding the small-team process; a framework does not remove coordination complexity.

Key terms

Term	Working definition
Software development process	An ordered model of engineering activities that creates shared procedures and control points.
Sequential process	Long-term upfront planning followed by phases completed largely in order, with deviations discouraged.
Iterative process	Small cycles that plan only what is needed, embrace change, and repeatedly inspect and adapt.
Waterfall	A linear sequence in which one activity is completed before the next begins; the deck no longer considers it generally viable.
V-Model	A sequential model pairing each development activity with a corresponding verification or test activity.
DevOps loop	A continuous connection between planning, building, delivering, operating, and learning rather than a handoff to one 'DevOps person'.

Term	Working definition
Agile	A value orientation favouring interaction, working software, customer collaboration, and response to change while still using necessary tools, documentation, contracts, and plans.
Extreme Programming (XP)	An agile approach associated in the course with fast feedback and engineering practices such as TDD and pairing.
Scrum	An iterative framework defining roles, timeboxed Sprints, events, backlogs, and a reviewable product increment.
Kanban	A lean method that visualises work, limits work in progress, exposes bottlenecks, and optimises flow through pull.
Flow	The smooth movement of valuable work through a process to completion.
Utilisation	The proportion of available capacity that is occupied; maximum utilisation can create queues and slow flow.
Throughput	The rate at which a process creates completed output in a given period.
Work in Progress (WIP)	Started work that has not yet reached the agreed definition of completion.
Bottleneck	A constrained process step that limits overall flow.
Waste	Non-value-adding activity such as excess inventory/WIP, motion, waiting, over-processing, or defects.
Pull principle	A downstream step starts new work only when it has capacity, instead of work being pushed into a queue.
Definition of Ready (DoR)	A team checklist confirming that a story has enough value, clarity, quality targets, dependency knowledge, and manageable size to start.
Definition of Done (DoD)	A team checklist confirming that acceptance and quality criteria pass, regression protection exists, and the change has reached the agreed delivery state.
SAFe / LeSS	Frameworks named by the course for coordinating agile work at larger scale; the slides show the frameworks but do not prescribe one.

Important distinctions

- Testing asks whether the team solved the problem correctly; validation asks whether the team solved the right problem.
- Scrum structures iterative planning and review; Kanban focuses on continuous flow and WIP. Teams can combine pull principles with Scrum.
- Utilisation measures busyness; throughput measures completed output. They are not interchangeable.
- DoR controls entry into implementation; DoD controls exit from the delivery workflow.

Typical mistakes

- Equating agile with no plan, no documentation, or no process.
- Starting more work to make everyone busy while queues and cycle time grow.
- Treating the board as reporting furniture rather than a shared flow-control mechanism.
- Allowing each specialist to optimise local tasks instead of swarming on the bottleneck.
- Calling work done when it merely works on one machine or remains undeployed.

MentorHive application

SYNTHESIS For MentorHive, a two-week Sprint can provide a planning and review rhythm, while a pull-based board and WIP limits keep booking, payment, and notification work flowing. A story is Ready only when business value, acceptance criteria, quality scenarios, dependencies, and size are clear; it is Done only when those criteria pass, regression tests exist, and the feature reaches the agreed production state.

Exit criterion

CHECK YOURSELF Compare Waterfall, V-Model, Scrum, and Kanban for one scenario. Choose a process, justify the risk and feedback trade-offs, and design a board with WIP limits plus a DoR and DoD.

SOURCE TRAIL Course source: *SW Engineering - 06 - SW Dev Processes - 2026, slides/pages 3-46*. The course activity map is on slide 4; roles on 5-6; process models on 8-25; flow and waste on 24-35; DoR/DoD on 36-42; scaling on 43-46.

7. System architecture: decompose the technical system

COURSE DECK 07

System architecture covers the overall organisation of software, hardware, networks, storage, and other critical elements. Its purpose is to control complexity through explicit boundaries and relationships.

GUIDING QUESTION Which system boundaries, services, data responsibilities, and communication patterns best satisfy the quality goals and team structure?

Core logic

- Delay difficult-to-reverse choices until the last responsible moment, not until the last possible moment.
- Decompose the system into coherent sub-systems and externally visible services.
- Aim for high cohesion inside a boundary and loose coupling across boundaries.
- Choose layering, partitioning, data ownership, and connectors deliberately.
- Treat architectural patterns as reusable approaches that must be adapted to the problem, not copied as finished solutions.
- Compare build, framework, platform, cloud, and on-premise choices by cost, risk, flexibility, and time to market.
- Align team ownership and communication paths with the desired system boundaries.

Key terms

Term	Working definition
System architecture	The structure and behaviour of the complete technical system, including software, hardware, networks, storage, and other critical elements.
Last Responsible Moment	Delay a decision until the cost of not deciding becomes greater than the cost of deciding, preserving options without creating avoidable risk.
Pattern	A reusable, context-dependent solution approach that must be adapted before it becomes an implementation.
Decomposition	Identifying sub-systems, services, and their relationships to control complexity.
Sub-system	A collection of related classes, functions, events, and associations that work together.
Service	A group of externally visible operations provided through a stable interface.
Cohesion	The strength of relationships within a sub-system; high cohesion groups related responsibilities.

Term	Working definition
Coupling	The strength of dependencies between sub-systems; loose coupling limits the effects of internal change.
Layering	Hierarchical decomposition where lower layers provide services to higher layers.
Partitioning	Decomposition into peer sub-systems responsible for different classes of service.
Component	A computational unit such as a sub-system, database, layer, or object with a defined interface.
Connector	The interaction mechanism or protocol between components, such as a method call, event, or shared data.
COTS / Make-or-Buy	A decision about reusing libraries, frameworks, or platforms rather than creating every capability from scratch.

Pattern reference

Pattern names and core descriptions are course-sourced. Decision guidance and trade-off wording are synthesis prompts; validate them against the concrete context.

Pattern	Core idea	Primary strength	Primary cost / risk
Layered	Hierarchy of responsibilities	Reuse, standardisation, low coupling, testability	Indirect calls, setup effort, lower-layer change may propagate
N-tier	Layers deployed across two or more tiers	Separates client, services, business logic, storage	More distribution and operational complexity
SOA	Black-boxed services representing business activities	Composition and interoperable service communication	Governance and integration overhead
REST	Stateless services with uniform resource-oriented constraints, often over HTTP	Simple web service interactions and cacheability	Constraints must be applied consistently; not every HTTP API is RESTful
Microservices	Small business-capability services in separate processes and independently deployable	Strong boundaries, independent deployment, technology diversity	Distribution, consistency, and operational complexity
Self-Contained Systems	Autonomous web applications owning UI, logic, and data for a domain	Team autonomy with less overhead than very fine-grained services	Distribution and cross-system consistency remain
Event-Driven	Entities produce and consume events through messaging, routing, and optional event stores	Loose temporal coupling and reaction to state change	Traceability, ordering, consistency, and debugging complexity
ETL	Extract data, transform it into business entities, and load a data warehouse	Data integration and analytical preparation	Batch latency, data quality, and pipeline ownership
MVC	Separates model, view, and controller in a user-interface architecture	Decouples data from presentation and supports UI change	Controller complexity and variant-specific trade-offs

Important distinctions

- System architecture includes the complete technical environment; software architecture focuses on the high-level software structure.
- Layering is hierarchical; partitioning creates peer boundaries at a similar abstraction level.
- SOA is a broad service orientation; REST is a constraint style for services; Microservices are a specialised deployment and ownership form of SOA in the course.
- A pattern is an adaptable approach; an algorithm is a concrete finite procedure for a specific problem.

Typical mistakes

- Choosing Microservices because they sound modern without an independent-deployment or team-boundary need.
- Creating low-cohesion modules that group unrelated work or highly coupled services that cannot change independently.
- Calling any HTTP interface RESTful without examining statelessness, uniformity, separation, cacheability, and composability.
- Ignoring data ownership and consistency when splitting the system.
- Making every architecture choice early instead of preserving reversible options.

MentorHive application

SYNTHESIS A reasonable starting synthesis for MentorHive is a layered application partitioned into matching, scheduling, session, and payment modules with explicit interfaces. A calendar provider can be bought and integrated through an Adapter; independently deployable services should be introduced only when quality or ownership needs justify their distribution cost.

Exit criterion

CHECK YOURSELF Create a system context and decomposition for one product. Evaluate at least three patterns against quality scenarios, team boundaries, data ownership, and operational cost; explain why the fashionable option may not be the best option.

SOURCE TRAIL Course source: SW Engineering - 07 - System Architecture - 2026, slides/pages 2-26. Pattern descriptions are explicit; 'use when' guidance and the MentorHive starting point are synthesis.

8. Software architecture: structure, patterns, and decisions

COURSE DECK 08

Software architecture defines the high-level structure of the software and the discipline of creating it. Programming paradigms, design patterns, documentation, and decision records make that structure understandable and changeable.

GUIDING QUESTION How should software responsibilities and interactions be structured, communicated, and recorded so future changes remain possible?

Core logic

- Choose language paradigms with awareness of how they shape the design space.
- Use object-oriented principles to separate concerns and create meaningful, maintainable modules when OOP is the chosen paradigm.
- Apply design patterns to recurring code-structure and interaction problems, while accounting for their costs.
- Use UML views and arc42 to communicate architecture from multiple perspectives.
- Record important decisions in ADRs close to the code, including negative and neutral consequences.
- Invest in architecture according to product lifetime and change pressure; good design has an economic payoff over time.

Key terms

Term	Working definition
Software architecture	The high-level structure of a software system and the discipline of creating that structure.
Object	A concrete instance of a class with state and behaviour.
Class	A blueprint defining the structure and behaviour of objects.
Encapsulation	Grouping related data and behaviour behind a meaningful boundary.
Inheritance	Reusing and specialising structures and behaviour through a class hierarchy.
Polymorphism	Using a shared abstraction while concrete subtypes provide variant behaviour.
Abstraction	Defining a common contract or base while hiding or postponing concrete details.
Interface	A contract that an implementing class follows; a class may implement multiple interfaces.

Term	Working definition
Design pattern	A reusable approach to a recurring object creation, structure, or communication problem.
UML	A family of structure and behaviour diagrams used to communicate software and system views.
arc42	A framework for structuring architecture documentation.
Architecture Decision Record (ADR)	A durable record of an important decision: title, context, decision, status, and all consequences.
Design Stamina Hypothesis	A model in which good design starts more slowly but later supports faster cumulative functionality as change continues.

Pattern reference

Pattern names and core descriptions are course-sourced. Decision guidance and trade-off wording are synthesis prompts; validate them against the concrete context.

Pattern	Purpose	Main caution
Singleton	Restricts object creation to one instance	Global state and hidden coupling; use sparingly
Adapter	Wraps an incompatible type so it can be used through the expected interface	Additional translation layer
Facade	Provides a simpler interface over a more complex subsystem	Can become a broad, overly central gateway
Mediator	Centralises interaction logic so collaborating objects are less directly coupled	Mediator itself can become complex
Observer	Notifies registered observers about events; also called publish/subscribe in the deck	Event flow can become hard to trace
Strategy	Selects or changes an algorithm through a common interface at runtime	More objects and configuration

Important distinctions

- Generalisation consolidates shared behaviour; specialisation adds behaviour unique to a subtype.
- Encapsulation creates boundaries; abstraction defines what matters while hiding implementation detail.
- UML provides diagram types; arc42 provides a documentation structure; an ADR records one decision and its consequences.
- An ADR status can be proposed, accepted, deprecated, or superseded. A changed decision should remain traceable rather than disappear.

Typical mistakes

- Using inheritance only for code reuse even when the 'is-a' relationship is invalid.
- Treating a pattern name as proof that a design is appropriate.
- Using Singleton casually and introducing global state.
- Documenting only the chosen option and its benefits, while omitting forces, alternatives, and negative consequences.
- Creating architecture documents far from the code where they become undiscoverable and stale.

MentorHive application

SYNTHESIS MentorHive can use an Adapter for an external calendar API, Strategy for selectable matching algorithms, Observer for booking notifications, and a Facade to present a simple booking interface over scheduling, payment, and messaging. An ADR should explain why the pattern or integration was chosen, which alternatives were considered, and which complexity was accepted.

Exit criterion

CHECK YOURSELF Model one feature with a UML component or class view and write a complete ADR. Apply one design pattern, explain the problem it solves, and identify the new risk or coupling it introduces.

SOURCE TRAIL Course source: *SW Engineering - 08 - Software Architecture - 2026*, slides/pages 2-33. OOP is on 4-17; patterns on 18-25; UML, arc42, ADRs, and architecture economics on 26-33.

9. Design and implementation: build quality in

COURSE DECK 09

Implementation is broader than writing code. It includes UI/UX, infrastructure, frameworks, libraries, configuration, tooling, tests, pipelines, deployment choices, and documentation. Quality must be created during this work.

GUIDING QUESTION How can the team turn requirements and architecture into usable software while receiving fast, reliable feedback?

Core logic

- Clarify whether 'design' means architecture or the user interface and experience.
- Translate stories into interface concepts with reusable components, a design system, and iterative critique.
- Choose implementation technologies and tools as part of the solution, not as coding trivia.
- Use TDD to derive a test from the requirement, write enough code to pass, then refactor without changing behaviour.
- Balance many fast isolated tests with fewer integrated tests for whole-system confidence.
- Use test doubles to isolate dependencies and BDD scenarios to connect acceptance criteria to executable checks.
- Combine fault avoidance, fault detection, and fault tolerance.
- Use review, pairing, or ensemble work to move feedback earlier and share knowledge.

Key terms

Term	Working definition
UI/UX design	The creation of the software's look, feel, interactions, and user workflows.
UI component	A reusable interaction element such as a label, button, table, list, view, or layout.
Design system	Standards, guidelines, and assets defining a consistent visual and experiential language.
Component library	Implemented reusable UI components that embody a design system.
Design Studio Method	A short iterative process: align on a story, sketch individually, pitch and critique, then repeat and improve.
Syntax	The grammar and validity rules of a programming language.
Semantics	The meaning and behaviour of a program statement.

Term	Working definition
Integrated Development Environment (IDE)	Tooling that supports coding, highlighting, debugging, framework management, and multiple languages.
Debugging	Finding, analysing, and fixing defects in software.
Test-Driven Development (TDD)	Creating a test before the solution, writing code that passes it, and refactoring under test protection.
Unit / Component Test	A test of a small piece of code in isolation.
Service / Integration Test	A test of two or more units working together.
UI / End-to-End Test	A test of the integrated application through a user-facing or system-wide path.
Testing Pyramid	A heuristic favouring many fast isolated tests and fewer slower integrated tests; terminology and boundaries vary.
Test Double	A replacement for a real dependency during a test.
Acceptance Test	A test checking whether the implemented system fulfils requirements; it may be manual or automated.
Regression Test	A repeated test protecting existing behaviour after change.
Behaviour-Driven Development (BDD)	Writing behavioural acceptance scenarios before implementation and connecting them to executable checks.
Four-eye principle	Involving another engineering perspective to obtain feedback and reduce costly rework.
Pair Programming	Driver and navigator work on the same task and switch roles, creating immediate review and learning.
Ensemble Programming	The whole team works on one task with rotating roles, useful for critical multidisciplinary problems.

Important distinctions

- Compilation and type checks identify invalid structure; semantic defects require analysis and behavioural tests.
- Acceptance tests execute against an implemented system, but BDD scenarios can and should be written before implementation.
- Isolated tests are fast and easy to diagnose; integrated tests provide stronger confidence that parts work together.
- Code review is often asynchronous and later; pairing and ensemble work shift feedback into the act of implementation.

Typical mistakes

- Treating design as an unqualified term and confusing UI/UX with architecture.
- Equating implementation with typing code while ignoring configuration, dependencies, deployment, and documentation.
- Turning the Testing Pyramid into dogma or inverting it into a slow, brittle set dominated by end-to-end tests.
- Assuming automation removes the need for exploratory or user testing.
- Calling a test double a mock or stub without specifying the actual behaviour; the deck only defines the broader replacement concept.
- Using code review as a late quality gate that creates waiting and rework rather than fast learning.

MentorHive application

SYNTHESIS For MentorHive slot booking, write a failing test for the rule 'a slot can be booked once', implement the smallest reservation logic that passes, then refactor. Replace the calendar and notification services with test doubles for unit tests; use service tests for database concurrency and a small number of end-to-end scenarios for the complete learner journey.

Exit criterion

CHECK YOURSELF Design a test strategy for one story: map risks to unit, integration, and end-to-end tests; identify required doubles, one BDD scenario, fault-handling mechanisms, and the review mode that provides the best feedback timing.

SOURCE TRAIL Course source: *SW Engineering - 09 - Design & Implementation - 2026*, slides/pages 2-34. UI/UX is on 2-8; implementation on 9-17; quality during implementation on 18-29; four-eye practices on 30-34.

10. Builds and releases: make change traceable and releasable

COURSE DECK 10

A delivery system turns locally working code into a repeatable, traceable, approved, and deployable change. Automation reduces manual variation and creates faster integration and production feedback.

GUIDING QUESTION How can small changes be integrated, tested, versioned, promoted, and measured without large merge or release events?

Core logic

- Track code, infrastructure, configuration, and database change in version control where possible.
- Integrate small, logically connected commits frequently and keep local work current.
- Automate compilation, dependency resolution, tests, packaging, and promotion consistently on local and remote systems.
- Use branches only for a clear isolation need and keep them short to reduce deferred integration.
- Keep the main branch releasable through Continuous Integration and automated quality gates.
- Distinguish Continuous Delivery from automatic production deployment.
- Use feature toggles to integrate unfinished code without executing it, while preventing untested behaviour in production.
- Measure the delivery system through DORA metrics, never through simplistic individual output counts.

Key terms

Term	Working definition
Developer platform	A platform combining version control, automated actions, package build and release, and deployment-pipeline capabilities.
Build automation	Repeatable compilation, dependency resolution, testing, and packaging driven by scripts or a pipeline.
Deployment pipeline	An automated sequence that moves a change through build, tests, staging, production, and post-deployment feedback.
Version Control System	A system for collaborative change tracking, commit history, and restoration of earlier states.
Merge	The integration of code changes into a shared code base.
Commit	A logically selected set of staged changes saved in the local repository.

Term	Working definition
Push / Pull	Push uploads local commits; pull fetches and merges remote changes.
Version	A specific state of an application or library.
Release	A formally approved, tested, validated, documented, and accepted version.
Commit hash	An internal unique identifier for a repository state.
Tag	A named pointer to a specific commit hash.
Semantic Versioning (SemVer)	MAJOR.MINOR.PATCH: incompatible API change, backward-compatible feature, backward-compatible fix.
Branch	A separate development timeline that enables parallel or versioned work but defers later integration.
Continuous Integration (CI)	Integrating changes into the main branch regularly, multiple times per day.
Continuous Delivery	Keeping the main branch releasable through automation; production release may remain a manual decision.
Continuous Deployment	Automatically deploying a releasable main branch to production.
Feature Toggle	A switch that allows code to be integrated while its behaviour remains disabled until it is safe to activate.
DORA metrics	Four system-level measures covering release frequency, lead time, production failure rate, and restoration speed.

DORA metrics

Metric	System-level meaning
Deployment Frequency	How often the organisation successfully releases to production.
Lead Time for Changes	Time from a commit to its production deployment.
Change Failure Rate	Percentage of deployments that cause a production failure.
Time to Restore Service	Time required to recover from a production failure.

Important distinctions

- A commit changes the local repository; a push transfers commits to the remote repository.
- A version is any identified state; a release has passed formal acceptance and approval.
- Continuous Delivery means ready to release; Continuous Deployment means automatic release to production.
- Branches and feature toggles are compatible tools, but neither permits untested behaviour to execute in production.

- DORA balances speed and stability at system level; it is not an individual productivity scorecard.

Typical mistakes

- Large mixed commits that are hard to review, test, or revert.
- Long-lived branches and big-bang merges.
- Assuming Git Flow is always best practice; the deck frames its complexity as overhead when extensive manual regression testing is not required.
- Calling occasional builds CI without frequent main-branch integration.
- Leaving feature toggles unmanaged until they become permanent complexity.
- Measuring engineers by lines of code, commit count, or isolated individual goals that damage collaboration.

MentorHive application

SYNTHESIS MentorHive integrates the booking feature in small commits behind a disabled toggle. The pipeline compiles, runs unit and service tests, deploys to a stage, executes behavioural tests, promotes the approved version, then runs post-deployment checks. The toggle is enabled gradually only after the production signals are healthy.

Exit criterion

CHECK YOURSELF Draw a complete pipeline from working directory to production. Explain the state transitions, quality gates, Delivery-versus-Deployment decision, toggle strategy, and how each DORA metric would be measured.

SOURCE TRAIL Course source: *SW Engineering - 10 - Builds and Releases - 2026, slides/pages 2-33. Platform and automation are on 2-4; version control on 5-14; versions on 15-16; branching on 17-20; CI/CD on 21-25; measurement on 26-33.*

11. Operations and maintenance: learn from reality

COURSE DECK 11

Production is where assumptions meet reality. Teams need ownership, useful signals, blameless incident learning, and disciplined maintenance to keep the system reliable and changeable.

GUIDING QUESTION How do we detect and understand production behaviour, recover safely, prevent recurrence, and improve the system without losing delivery capacity?

Core logic

- Accept operational ownership: 'You build it, you run it.'
- Restore service first, then conduct a blameless Post-Mortem while memory is fresh.
- Analyse systemic causes and produce owned action items rather than assigning personal blame.
- Monitor known signals and create actionable alerts with meaningful thresholds, duration, severity, and context.
- Build observability so the team can ask new questions and reconstruct internal system state.
- Prefer targeted, incremental refactoring when it can preserve behaviour and reduce risk; treat a rewrite as a major new-system decision.
- Identify and repay technical debt before its interest slows business change and increases defects.
- Feed operational evidence back into quality scenarios, requirements, architecture, tests, and delivery work.

Key terms

Term	Working definition
Post-Mortem Analysis (PMA)	A systematic, blameless analysis shortly after an incident is resolved, resulting in a written record and action items.
Monitoring	Tracking predefined metrics, logs, traces, and conditions to answer known questions and alert on expected failure modes.
Alert	A notification triggered by a condition or event and carrying enough context to start a response.
Threshold	A value or condition separating expected from abnormal behaviour.
Static threshold	A fixed boundary, such as memory use above 80%.
Dynamic threshold	A boundary adapted to normal historical ranges and trends.
Anomaly	A deviation from an expected pattern.

Term	Working definition
Observability	The ability to understand a system through its outputs and ask new, open-ended questions about internal state.
Legacy code	Code whose ownership has been inherited by someone who did not originally write it; the course does not define it simply as old or bad code.
Refactoring	Improving internal structure without changing external behaviour.
Rewrite	Starting again and reopening architecture, technology, feature, and test choices.
Hot Spot Refactoring	Prioritising code with high change frequency, complexity, or bug concentration.
Campground Rule Refactoring	Using a technical vision and proof of concept, then improving touched areas incrementally.
Preparatory Refactoring	Improving structure before a feature because the current architecture makes the change unnecessarily difficult.
Technical Debt	Shortcuts or erosion that make architecture and quality less fit, increasing future change cost and defect risk.
Technical Excellence	The course's stated opposite of technical debt: sustained investment in quality and improvement.

Important distinctions

- Monitoring answers predefined questions; observability supports new questions when the failure was not anticipated.
- Blameless does not mean consequence-free: the PMA still documents impact, causes, lessons, and concrete action items.
- Refactoring preserves external behaviour; a rewrite creates a new implementation and often a new architecture.
- Technical debt is not only visible bugs; it includes architecture mismatch, missing tests, unmet quality targets, and outdated dependencies.

Typical mistakes

- Running the PMA during the active incident instead of restoring service first.
- Producing alerts that are noisy, unactionable, or missing ownership and context.
- Treating dashboards as observability without the ability to investigate new questions.
- Assuming a rewrite is automatically cleaner, faster, or safer than incremental improvement.
- Allowing PMA action items to remain unowned or invisible to the delivery backlog.
- Accepting technical debt without recording the reason, interest, and repayment path.

MentorHive application

SYNTHESIS In the MentorHive launch scenario, QA, Engineering, Product, and Marketing relied on assumptions about postponement, feature-flag state, and readiness. A blameless PMA would document more than 12,000 failed user journeys, an eightfold support spike, detection and resolution, the communication and release-control causes, and owned actions such as an explicit go/no-go owner and a pipeline-linked marketing readiness gate.

Exit criterion

CHECK YOURSELF Analyse one incident with a complete PMA, propose one actionable alert and one observability question, then choose a refactoring strategy and explain how it reduces a documented technical-debt interest cost.

SOURCE TRAIL Course source: *SW Engineering - 11 - Ops & Maintenance - 2026, slides/pages 2-18. PMA is on 2-7; monitoring and observability on 8-12; legacy code, refactoring, and debt on 13-18.*

12. End-to-end case: MentorHive

MentorHive is the recurring product scenario in the quality, requirements, and operations decks. The following route extends that course case across every topic so the relationships become visible.

STATUS The product premise and launch incident are course-sourced. The complete end-to-end solution path is synthesis and deliberately avoids presenting one architecture or process as universally correct.

1. Product intent

Help learners find relevant mentors and book useful sessions with low administrative friction.

2. Quality

Prioritise performance, reliability, security, interaction capability, and maintainability; write measurable scenarios and trade-offs.

3. Requirements

Create the booking User Story, map rules and edge cases, resolve time-zone and payment questions, and define pass/fail acceptance criteria.

4. Organisation

Use a cross-functional product team with explicit ownership of the complete learner journey and launch decision.

5. Process

Combine an iterative review rhythm with pull, WIP limits, a clear DoR, and a production-level DoD.

6. System architecture

Decompose matching, scheduling, session, payment, and notification responsibilities; select service and data boundaries only where justified.

7. Software architecture

Use an Adapter for a calendar provider, Strategy for matching algorithms, Observer for notifications, and ADRs for important choices.

8. Implementation

Apply TDD to double-booking rules, test doubles for external systems, integration tests for concurrency, and a small end-to-end suite.

9. Build and release

Integrate small changes through CI, keep main releasable, deploy behind a toggle, and observe post-deployment health.

10. Operate and evolve

Alert on booking failures and latency, use observability for unknown failure modes, run blameless PMAs, and repay technical debt through targeted refactoring.

13. High-value comparisons

Use these sheets when adjacent terms seem similar. They compress relationships that are spread across several decks.

Functional requirement vs quality requirement vs acceptance criterion

Concept	Core question / idea	Artefact / context	Example or trade-off
Functional Requirement	What behaviour or feature is needed?	User Story	As a learner, I want to book a mentor slot so that I can receive guidance.
Non-functional Requirement	How well must the behaviour work?	Quality scenario	During peak load, 95% of mentor searches return within two seconds.
Acceptance Criterion	What observable evidence proves the story is complete?	Pass/fail rule or Gherkin	Given a free slot, when the learner confirms, then one booking is created and both parties are notified.

Sequential and iterative process choices

Concept	Core question / idea	Artefact / context	Example or trade-off
Waterfall	Linear phase sequence	Stable, heavily controlled context where late feedback can be tolerated	Change and defects are discovered late; deck no longer considers it generally viable
V-Model	Each development activity has a corresponding test activity	Industrial or military contexts requiring explicit verification	Still sequential; failed tests force return through earlier phases
Scrum	Timeboxed iterative planning and review	Product work needing a regular goal and feedback rhythm	Can become ceremony without flow or engineering quality
Kanban	Pull, WIP limits, bottleneck control	Continuous demand and flow optimisation	Requires discipline to finish work and act on blocked flow

System architecture vs software architecture

Concept	Meaning	Key implication
Boundary	Software, hardware, networks, storage, deployment	Software components, classes, modules, interfaces
Typical questions	Where does it run? How is it split and connected? Who owns data and services?	How is code structured? Which paradigm or pattern? How are decisions documented?
Core artefacts	Context, decomposition, component/connector view, pattern selection	UML views, arc42 documentation, ADRs, design-pattern application

Concept	Meaning	Key implication
Shared goal	High cohesion, loose coupling, quality fit, understandable consequences	High cohesion, loose coupling, quality fit, understandable consequences

Test levels and feedback

Concept	Core question / idea	Artefact / context	Example or trade-off
Unit / Component	Small piece in isolation	Fast, precise, easy to change	Limited confidence in integration
Service / Integration	Two or more units together	Checks contracts and collaboration	Slower and more complex setup
UI / End-to-End	Integrated user or system path	High confidence in complete behaviour	Slow, brittle, and harder to diagnose
Acceptance	Requirement fulfilment	Connects product agreement to evidence	May be manual or automated; scope must remain outcome-focused

CI, Continuous Delivery, and Continuous Deployment

Concept	Meaning	Key implication
Continuous Integration	Merge into main multiple times per day	Small integration batches and fast conflict/test feedback
Continuous Delivery	Main is always releasable	Automated tests and promotion; production decision may be manual
Continuous Deployment	Every releasable change is deployed automatically	Fully automated promotion to production plus post-deploy feedback

Monitoring, observability, and incident learning

Concept	Meaning	Key implication
Monitoring	Are known signals inside expected boundaries?	Metrics, logs, traces, dashboards, predefined alerts
Observability	What is happening internally, including an unanticipated problem?	Explore outputs and correlate evidence to reconstruct state
Post-Mortem	Why did the incident occur and how will the system change?	Blameless timeline, impact, causes, lessons, and action items

14. Video Club: targeted depth

The Video Club PDF supplies links, titles, and reflection prompts rather than transcripts. Use it to deepen a topic after the relevant handbook chapter; do not infer speaker positions from a title alone.

Theme	Speaker(s)	Connects to	Slides
Role of testers / Whole Team Quality	Janet Gregory and Lisa Crispin	Quality ownership and testing across the lifecycle	Slides 2-3
Holistic Testing	Janet Gregory and Lisa Crispin	Quality activities, models, and whole-team thinking	Slides 4-6
An Introduction to Continuous Delivery	Dave Farley	Delivery flow and automation	Slides 7-8
Making Architecture Matter	Martin Fowler	Architecture economics and long-term change	Slides 9-11
TDD isn't hard, it's something else	Dave Farley	Test-first implementation and feedback	Slides 12-13
The BEST Developer Productivity Metrics We Have... SO FAR	Dave Farley and Martin Fowler	System-level productivity and measurement	Slides 14-15

SUGGESTED METHOD Before viewing, write your current explanation. During the video, record unfamiliar terms and claims. Afterwards, compare the speaker's model with the course model and identify one agreement, one tension, and one practical implication.

15. Alphabetical glossary

Definitions are intentionally concise. Use the chapter pointer for the model, distinctions, examples, and source trail.

Term	Concise definition	See
Abstraction	Defining a common contract or base while hiding or postponing concrete details.	Chapter 8
Acceptance Criterion	A specific, outcome-focused condition that can be verified as pass or fail and describes when a story is done.	Chapter 3
Acceptance Test	A test checking whether the implemented system fulfils requirements; it may be manual or automated.	Chapter 9
Agile	A value orientation favouring interaction, working software, customer collaboration, and response to change while still using necessary tools, documentation, contracts, and plans.	Chapter 6
Alert	A notification triggered by a condition or event and carrying enough context to start a response.	Chapter 11
Anomaly	A deviation from an expected pattern.	Chapter 11
arc42	A framework for structuring architecture documentation.	Chapter 8
Architecture Decision Record (ADR)	A durable record of an important decision: title, context, decision, status, and all consequences.	Chapter 8
Backend	Server-side logic, data access, and service interfaces.	Chapter 1
Behaviour-Driven Development (BDD)	Writing behavioural acceptance scenarios before implementation and connecting them to executable checks.	Chapter 9
Bottleneck	A constrained process step that limits overall flow.	Chapter 6
Branch	A separate development timeline that enables parallel or versioned work but defers later integration.	Chapter 10
Build automation	Repeatable compilation, dependency resolution, testing, and packaging driven by scripts or a pipeline.	Chapter 10
Campground Rule Refactoring	Using a technical vision and proof of concept, then improving touched areas incrementally.	Chapter 11
Change Failure Rate	Percentage of deployments that cause a production failure.	Chapter 10
Class	A blueprint defining the structure and behaviour of objects.	Chapter 8
Cloud server	Remotely provisioned computing resources operated through a cloud environment.	Chapter 1
Cohesion	The strength of relationships within a sub-system; high cohesion groups related responsibilities.	Chapter 7
Commit	A logically selected set of staged changes saved in the local repository.	Chapter 10
Commit hash	An internal unique identifier for a repository state.	Chapter 10
Compatibility	The product co-exists and exchanges usable information with other systems.	Chapter 2
Component	A computational unit such as a sub-system, database, layer, or object with a defined interface.	Chapter 7

Term	Concise definition	See
Component library	Implemented reusable UI components that embody a design system.	Chapter 9
Connector	The interaction mechanism or protocol between components, such as a method call, event, or shared data.	Chapter 7
Continuous Delivery	Keeping the main branch releasable through automation; production release may remain a manual decision.	Chapter 10
Continuous Deployment	Automatically deploying a releasable main branch to production.	Chapter 10
Continuous Integration (CI)	Integrating changes into the main branch regularly, multiple times per day.	Chapter 10
Conway's Law	Systems tend to reflect the communication structures of the organisations that design them.	Chapter 5
COTS / Make-or-Buy	A decision about reusing libraries, frameworks, or platforms rather than creating every capability from scratch.	Chapter 7
Coupling	The strength of dependencies between sub-systems; loose coupling limits the effects of internal change.	Chapter 7
Database	Persistent storage and retrieval of application data.	Chapter 1
Debugging	Finding, analysing, and fixing defects in software.	Chapter 9
Decomposition	Identifying sub-systems, services, and their relationships to control complexity.	Chapter 7
Definition of Done (DoD)	A team checklist confirming that acceptance and quality criteria pass, regression protection exists, and the change has reached the agreed delivery state.	Chapter 6
Definition of Ready (DoR)	A team checklist confirming that a story has enough value, clarity, quality targets, dependency knowledge, and manageable size to start.	Chapter 6
Deployment Frequency	How often the organisation successfully releases to production.	Chapter 10
Deployment pipeline	An automated sequence that moves a change through build, tests, staging, production, and post-deployment feedback.	Chapter 10
Design pattern	A reusable approach to a recurring object creation, structure, or communication problem.	Chapter 8
Design Stamina Hypothesis	A model in which good design starts more slowly but later supports faster cumulative functionality as change continues.	Chapter 8
Design Studio Method	A short iterative process: align on a story, sketch individually, pitch and critique, then repeat and improve.	Chapter 9
Design system	Standards, guidelines, and assets defining a consistent visual and experiential language.	Chapter 9
Developer platform	A platform combining version control, automated actions, package build and release, and deployment-pipeline capabilities.	Chapter 10
DevOps loop	A continuous connection between planning, building, delivering, operating, and learning rather than a handoff to one 'DevOps person'.	Chapter 6
DORA metrics	Four system-level measures covering release frequency, lead time, production failure rate, and restoration speed.	Chapter 10
Dynamic threshold	A boundary adapted to normal historical ranges and trends.	Chapter 11
Encapsulation	Grouping related data and behaviour behind a meaningful boundary.	Chapter 8
Ensemble Programming	The whole team works on one task with rotating roles, useful for critical multidisciplinary problems.	Chapter 9

Term	Concise definition	See
Example	A concrete case illustrating how a rule applies, including boundaries and exceptions.	Chapter 3
Example Mapping	A collaborative method that links one story to rules, examples, open questions, and newly discovered stories.	Chapter 3
Extreme Programming (XP)	An agile approach associated in the course with fast feedback and engineering practices such as TDD and pairing.	Chapter 6
Feature Toggle	A switch that allows code to be integrated while its behaviour remains disabled until it is safe to activate.	Chapter 10
Flexibility	The product can adapt, scale, install, and replace components across changing contexts and environments.	Chapter 2
Flow	The smooth movement of valuable work through a process to completion.	Chapter 6
Four-eye principle	Involving another engineering perspective to obtain feedback and reduce costly rework.	Chapter 9
Frontend	Client-side software that presents information and handles interaction.	Chapter 1
Functional organisation	People are grouped by specialist department, supporting expertise but creating cross-department handoffs.	Chapter 5
Functional Requirement (FR)	A required feature or behaviour of the system.	Chapter 3
Functional Suitability	Functions cover the intended tasks, produce correct results, and support user objectives appropriately.	Chapter 2
Gherkin	A Given-When-Then format: Given context, When an event occurs, Then an observable outcome follows.	Chapter 3
Holacracy	An organisation of autonomous circles and explicit roles connected through governance roles.	Chapter 5
Hot Spot Refactoring	Prioritising code with high change frequency, complexity, or bug concentration.	Chapter 11
Infrastructure	The compute, network, runtime, and deployment foundation on which software operates.	Chapter 1
Inheritance	Reusing and specialising structures and behaviour through a class hierarchy.	Chapter 8
Integrated Development Environment (IDE)	Tooling that supports coding, highlighting, debugging, framework management, and multiple languages.	Chapter 9
Interaction Capability	Specified users can understand and operate the system to complete tasks in their context of use; the slides also call it usability.	Chapter 2
Interface	A contract that an implementing class follows; a class may implement multiple interfaces.	Chapter 8
Iterative process	Small cycles that plan only what is needed, embrace change, and repeatedly inspect and adapt.	Chapter 6
Kanban	A lean method that visualises work, limits work in progress, exposes bottlenecks, and optimises flow through pull.	Chapter 6
Last Responsible Moment	Delay a decision until the cost of not deciding becomes greater than the cost of deciding, preserving options without creating avoidable risk.	Chapter 7
Layering	Hierarchical decomposition where lower layers provide services to higher layers.	Chapter 7
Lead Time for Changes	Time from a commit to its production deployment.	Chapter 10

Term	Concise definition	See
Legacy code	Code whose ownership has been inherited by someone who did not originally write it; the course does not define it simply as old or bad code.	Chapter 11
Maintainability	The system can be analysed, tested, modified, and reused effectively without excessive side effects.	Chapter 2
Maintenance	Correcting, adapting, and evolving software after delivery.	Chapter 1
Matrix organisation	People retain a functional home while contributing to one or more product or project teams, creating dual alignment.	Chapter 5
Merge	The integration of code changes into a shared code base.	Chapter 10
Monitoring	Tracking predefined metrics, logs, traces, and conditions to answer known questions and alert on expected failure modes.	Chapter 11
Non-functional Requirement (NFR)	A measurable quality expectation for the system's behaviour.	Chapter 3
Object	A concrete instance of a class with state and behaviour.	Chapter 8
Observability	The ability to understand a system through its outputs and ask new, open-ended questions about internal state.	Chapter 11
On-premise data centre	Computing facilities controlled at an organisation's own site.	Chapter 1
Open Question	An unresolved issue made visible during refinement and clarified no later than the start of development.	Chapter 3
Operations	Running, monitoring, and supporting software in production.	Chapter 1
Overengineering	Adding more complexity, capacity, or quality than the context requires, thereby increasing cost and slowing change.	Chapter 2
Pair Programming	Driver and navigator work on the same task and switch roles, creating immediate review and learning.	Chapter 9
Partitioning	Decomposition into peer sub-systems responsible for different classes of service.	Chapter 7
Pattern	A reusable, context-dependent solution approach that must be adapted before it becomes an implementation.	Chapter 7
Performance Efficiency	Required response time, throughput, resource use, and capacity are achieved under specified conditions.	Chapter 2
Polymorphism	Using a shared abstraction while concrete subtypes provide variant behaviour.	Chapter 8
Post-Mortem Analysis (PMA)	A systematic, blameless analysis shortly after an incident is resolved, resulting in a written record and action items.	Chapter 11
Preparatory Refactoring	Improving structure before a feature because the current architecture makes the change unnecessarily difficult.	Chapter 11
Procedural organisation	The recurring processes and coordination rules through which work moves.	Chapter 5
Product	A value offering managed across its lifecycle from idea through development, launch, operation, and evolution.	Chapter 5
Project	A temporary, unique endeavour with a defined beginning and end, managed within constraints such as scope, time, and budget.	Chapter 5
Project Management Triangle	Scope, time, and budget constrain the work; uncompensated change in one dimension reduces quality or forces change in another.	Chapter 5

Term	Concise definition	See
Project organisation	People leave or cross their units to form a dedicated temporary delivery team.	Chapter 5
Pull principle	A downstream step starts new work only when it has capacity, instead of work being pushed into a queue.	Chapter 6
Push / Pull	Push uploads local commits; pull fetches and merges remote changes.	Chapter 10
Quality scenario	A measurable statement linking conditions and a stimulus to an observable response and quality characteristic.	Chapter 2
Quality trade-off	A deliberate choice that improves one quality characteristic while accepting a consequence in another.	Chapter 2
Refactoring	Improving internal structure without changing external behaviour.	Chapter 11
Refinement	Collaborative clarification, splitting, changing, or removal of requirements before implementation.	Chapter 3
Regression Test	A repeated test protecting existing behaviour after change.	Chapter 9
Release	A formally approved, tested, validated, documented, and accepted version.	Chapter 10
Reliability	The system performs required functions over time, remains available, tolerates faults, and can recover.	Chapter 2
Rewrite	Starting again and reopening architecture, technology, feature, and test choices.	Chapter 11
Rule	A condition or policy that governs how a User Story behaves.	Chapter 3
SAFe / LeSS	Frameworks named by the course for coordinating agile work at larger scale; the slides show the frameworks but do not prescribe one.	Chapter 6
Safety	The product avoids states that endanger life, health, property, or the environment.	Chapter 2
Scrum	An iterative framework defining roles, timeboxed Sprints, events, backlogs, and a reviewable product increment.	Chapter 6
Security	The system protects information and resists malicious action according to authorised access.	Chapter 2
Semantic Versioning (SemVer)	MAJOR.MINOR.PATCH: incompatible API change, backward-compatible feature, backward-compatible fix.	Chapter 10
Semantics	The meaning and behaviour of a program statement.	Chapter 9
Sequential process	Long-term upfront planning followed by phases completed largely in order, with deviations discouraged.	Chapter 6
Service	A group of externally visible operations provided through a stable interface.	Chapter 7
Service / Integration Test	A test of two or more units working together.	Chapter 9
Software architecture	The high-level structure of a software system and the discipline of creating that structure.	Chapter 8
Software development process	An ordered model of engineering activities that creates shared procedures and control points.	Chapter 6
Software engineering	An engineering discipline covering the decisions and activities required to build, deliver, run, and evolve software - not just programming.	Chapter 1
Specification waste	Detailed requirement work that becomes obsolete before the team needs to implement it.	Chapter 3

Term	Concise definition	See
Spotify model	A matrix vocabulary of autonomous Squads, capability-focused Chapters, collections of Squads called Tribes, and learning communities called Guilds.	Chapter 5
Static threshold	A fixed boundary, such as memory use above 80%.	Chapter 11
Structural organisation	The allocation of roles, responsibilities, hierarchy, positions, and reporting lines.	Chapter 5
Sub-system	A collection of related classes, functions, events, and associations that work together.	Chapter 7
Syntax	The grammar and validity rules of a programming language.	Chapter 9
System architecture	The structure and behaviour of the complete technical system, including software, hardware, networks, storage, and other critical elements.	Chapter 7
Tag	A named pointer to a specific commit hash.	Chapter 10
Team	People with a common goal who collaborate through complementary skills, communication, interdependence, and shared as well as individual accountability.	Chapter 5
Technical Debt	Shortcuts or erosion that make architecture and quality less fit, increasing future change cost and defect risk.	Chapter 11
Technical Excellence	The course's stated opposite of technical debt: sustained investment in quality and improvement.	Chapter 11
Test Double	A replacement for a real dependency during a test.	Chapter 9
Test-Driven Development (TDD)	Creating a test before the solution, writing code that passes it, and refactoring under test protection.	Chapter 9
Testing Pyramid	A heuristic favouring many fast isolated tests and fewer slower integrated tests; terminology and boundaries vary.	Chapter 9
Threshold	A value or condition separating expected from abnormal behaviour.	Chapter 11
Throughput	The rate at which a process creates completed output in a given period.	Chapter 6
Time to Restore Service	Time required to recover from a production failure.	Chapter 10
Tuckman model	A model of team development: Forming, Storming, Norming, Performing, and Adjourning; membership changes may reset the team.	Chapter 5
UI / End-to-End Test	A test of the integrated application through a user-facing or system-wide path.	Chapter 9
UI component	A reusable interaction element such as a label, button, table, list, view, or layout.	Chapter 9
UI/UX design	The creation of the software's look, feel, interactions, and user workflows.	Chapter 9
UML	A family of structure and behaviour diagrams used to communicate software and system views.	Chapter 8
Unit / Component Test	A test of a small piece of code in isolation.	Chapter 9
User Interface (UI)	The screens and controls through which a user interacts with a system.	Chapter 1
User Story	A concise requirement stated as role, desired action, and benefit: As [role], I want [action], so that [outcome].	Chapter 3
Utilisation	The proportion of available capacity that is occupied; maximum utilisation can create queues and slow flow.	Chapter 6
V-Model	A sequential model pairing each development activity with a corresponding verification or test activity.	Chapter 6

Term	Concise definition	See
Version	A specific state of an application or library.	Chapter 10
Version Control System	A system for collaborative change tracking, commit history, and restoration of earlier states.	Chapter 10
Waste	Non-value-adding activity such as excess inventory/WIP, motion, waiting, over-processing, or defects.	Chapter 6
Waterfall	A linear sequence in which one activity is completed before the next begins; the deck no longer considers it generally viable.	Chapter 6
Whole Team Quality	Quality is owned across roles and activities, not delegated to testers or a QA department.	Chapter 2
Work in Progress (WIP)	Started work that has not yet reached the agreed definition of completion.	Chapter 6

16. Source map and open information

The handbook is based on the complete local Software Engineering material set available on 6 August 2026. Original files remain unchanged.

Deck	File	Main content	Coverage
01	SW Engineering - 01 - Intro - 2026.pdf (12 pages)	History, prerequisites, layers, runtime environments, discipline scope	Slides 2-11
02	SW Engineering - 02 - Quality - 2026.pdf (27 pages)	Whole-team quality, ISO 25010, overengineering, trade-offs, quality scenarios	Slides 2-26
03	SW Engineering - 03 - Requirements Engineering - 2026.pdf (20 pages)	FR/NFR, iterative refinement, User Stories, Example Mapping, acceptance criteria	Slides 2-19
04	Not available (- pages)	Unknown	Open
05	SW Engineering - 05 - Organisational Structure - 2026.pdf (24 pages)	Teams, Tuckman, project/product, triangle, organisation forms	Slides 2-23
06	SW Engineering - 06 - SW Dev Processes - 2026.pdf (48 pages)	Activities, roles, sequential/iterative models, Scrum, Kanban, flow, DoR/DoD, scaling	Slides 3-46
07	SW Engineering - 07 - System Architecture - 2026.pdf (27 pages)	Decomposition, cohesion/coupling, architecture patterns, COTS, Conway's Law	Slides 2-26
08	SW Engineering - 08 - Software Architecture - 2026.pdf (34 pages)	OOP, design patterns, UML, arc42, ADRs, architecture economics	Slides 2-33
09	SW Engineering - 09 - Design & Implementation - 2026.pdf (35 pages)	UI/UX, implementation, TDD, Testing Pyramid, BDD, reviews and pairing	Slides 2-34
10	SW Engineering - 10 - Builds and Releases - 2026.pdf (34 pages)	Automation, Git, SemVer, branching, CI/CD, feature toggles, DORA	Slides 2-33
11	SW Engineering - 11 - Ops & Maintenance - 2026.pdf (19 pages)	PMA, monitoring, observability, legacy code, refactoring, technical debt	Slides 2-18
VC	SW Engineering - Video Club.pdf (16 pages)	Video prompts for quality, holistic testing, Delivery, architecture, TDD, metrics	Slides 2-15

Open course information

- Deck 04 and its topic are missing from the local source set.
- The assessment format, date, duration, language, permitted aids, weighting, and marking criteria are not stated in the available module files.
- The official module description and formal learning outcomes are not present in the module folder.
- The Video Club PDF contains links and prompts, not full transcripts; this handbook does not attribute arguments from videos that were not separately analysed.

BOUNDARY This handbook explains the available course content. It does not invent an assessment format, grading rule, deadline, or topic for the missing deck.

One-page final recall

- Quality tells us what 'good' means; requirements tell us what behaviour and outcomes are needed.
- Organisation and process determine communication, ownership, feedback speed, and flow.
- Architecture translates goals and constraints into boundaries and recorded decisions.
- Design, implementation, and tests create the executable solution and short feedback loops.
- Build and release automation make small changes traceable and safely releasable.
- Operations reveal reality; monitoring, observability, PMAs, and maintenance turn it into improvement.
- Every stage is connected by evidence, quality trade-offs, and feedback to earlier decisions.